into the sum, overwriting (and losing) all the values stored by the other thread(s). We lost data.

Fortunately, that's easy to fix. We use the built-in class Mutex (short for "mutually exclusive") to create synchronized regions—areas of code that only one thread may enter at a time.

Some grade schools coordinate students' hall access during class time using a system of hall passes. The number of passes is limited, and to leave the classroom, you need to take a pass with you. If someone else already has that pass, you have to wait for that person to return. The pass controls access to the shared resource—you have to own the pass to use the resource, and only one person can own it at a time.

A mutex is like that hall pass. You create a mutex to control access to a resource and then lock it when you want to use that resource. If no one else has it locked, your thread continues to run. If someone else has already locked that particular mutex, your thread suspends until they unlock it.

Here's a version of our counting code that uses a mutex to ensure that only one thread updates the count at a time:

tut_threads/mutex_1.rb

```
sum = 0
mutex = Thread::Mutex.new
threads = 10.times.map do
  Thread.new do
    100_000.times do
      # one at a time, please
      mutex.lock
      new_value = sum + 1
      print "#{new_value}  " if new_value % 250_000 == 0
      sum = new_value
      mutex.unlock
    end
  end
end
threads.each(&:join)
puts "\nsum = #{sum}"
```

produces:

```
250000  500000  750000  1000000
sum = 1000000
```

This pattern—lock a mutex, do something, then unlock—is so common that the Mutex class provides synchronize, which locks the mutex, runs the code in a block, and then unlocks the mutex. This also ensures that the mutex will get unlocked even if an exception is thrown while it is locked. Otherwise, an exception might cause the mutex to never unlock and permanently prevent other threads from gaining access to the shared resource.

tut_threads/mutex_1.rb

```
sum = 0
mutex = Thread::Mutex.new
threads = 10.times.map do
  Thread.new do
    100_000.times do
      # one at a time, please
      mutex.lock
```